

Digital Systems Project

Report Contents

Contents

I. Front Matter	3
1. Authors and Acknowledgements	3
2. Backup Strategy	3
II. Assignment 1 — Settable Watch	4
3. Development Cycle and Conventions	4
4. State Encoding	4
5. Per-Module Reports	4
5.1. seven_segment	4
5.2. up_down_counter	4
5.3. binary_to_bcd	5
5.4. hms_counter	5
5.5. mod_n_counter	5
5.6. restartable_rate_generator	5
5.7. top_time_display_v1	5
5.8. pwm_generator	6
5.9. rising_edge_detector	6
5.10. button_hold_detect	6
5.11. button_hold_pulse	6
5.12. button_auto_repeat	7
5.13. arming_latch	7
5.14. edit_mode_selector	7
5.15. editable_counter	7
5.16. key_synchroniser	7
5.17. decimal_display_driver	8
5.18. Board Wrapper (user_top / top_de1_soc)	8
5.19. Settable Watch V1–V4	8
6. Programming the Board	8

III. Assignment 2 — Stopwatch and Timer	9
7. Formal Verification	9
7.1. up_down_counter_rst	9
7.2. cascade_counter	9
8. Stopwatch	9
8.1. stopwatch_counter	10
8.2. snapshot_mux	10
8.3. stopwatch_control	10
8.4. user_top_stopwatch_v1	10
9. Timer	11
9.1. editable_countdown	11
9.2. user_top_timer_v1	11
10. Brightness Control	11
10.1. user_top_brightness_wrapper	11
11. Integration	11
11.1. user_top_timepiece_v1	11
11.2. user_top_brightness_timepiece	11
12. Synthesis (Yosys Walkthrough)	12
13. Enhancements (Optional)	12
14. Final Reflection	12
A. Cross-Check Index	13

Learning Objectives

- Know exactly what belongs in your report before you start writing it.
- Build the report incrementally, module by module, rather than reconstructing it from memory at the end.
- Cross-check that every **Report** box from both assignment sheets has been addressed.

Think First

This document shows every reporting requirement from Assignments 1 and 2 into the order you will actually build things, so you can fill it in as you go.

On numbering: Each box's heading states exactly which requirement it is and where it came from, e.g. **A1 Report 5**, matching the numbering in the original assignment PDFs directly.

Part I. Front Matter

1. Authors and Acknowledgements

Authors: *(everyone who wrote this report — you alone if sole-authored, or all co-authors if jointly authored)*

Acknowledgements: *(anyone else whose input benefited this report, even informally)*

2. Backup Strategy

A1 Report 3

Describe your backup strategy: version control with a remote repository, or dated copies on at least two other devices.

Part II.

Assignment 1 — Settable Watch

3. Development Cycle and Conventions

A1 Additional Oral Preparation 2

Be able to explain the development cycle and the project conventions in detail.

4. State Encoding

A1 Additional Oral Preparation 3

Be able to explain all four state encoding options for representing the current time, and justify why the three-counter encoding was chosen.

5. Per-Module Reports

Think First

Every module you build needs a waveform screenshot with a short explanation of why it is correct (A1 Report 2). This applies to every subsection below, whether or not it has additional questions.

5.1. `seven_segment`

A1 Report 2

Include a screenshot of the waveforms for this module and briefly explain why they are correct, pointing to specific features in the traces.

A1 Report 1

Explain why Verilator accepts `if (blank)` but not `if (ACTIVE_LOW)`.

5.2. `up_down_counter`

A1 Report 2

Include a screenshot of the waveforms for this module and briefly explain why they are correct, pointing to specific features in the traces.

5.3. binary_to_bcd

A1 Report 2

Include a screenshot of the waveforms for this module and briefly explain why they are correct, pointing to specific features in the traces.

A1 Report 4

Explain what binary-coded decimal (BCD) is.

5.4. hms_counter

A1 Report 2

Include a screenshot of the waveforms for this module and briefly explain why they are correct, pointing to specific features in the traces.

5.5. mod_n_counter

A1 Report 2

Include a screenshot of the waveforms for this module and briefly explain why they are correct, pointing to specific features in the traces.

5.6. restartable_rate_generator

A1 Report 2

Include a screenshot of the waveforms for this module and briefly explain why they are correct, pointing to specific features in the traces.

A1 Report 5

Explain how `localparam CountWidth = $clog2(CYCLE_COUNT);` works.

A1 Report 6

In addition to running the provided test bench, write your own test bench verifying correct behaviour when `CYCLE_COUNT = 1`. Include and explain it here.

5.7. top_time_display_v1

A1 Report 2

Include a screenshot of the waveforms for this module and briefly explain why they are correct, pointing to specific features in the traces.

5.8. pwm_generator

A1 Report 2

Include a screenshot of the waveforms for this module and briefly explain why they are correct, pointing to specific features in the traces.

A1 Report 7

Explain why this module is called a PWM generator.

A1 Report 8

In `mod_n_counter`, `rst` takes priority over `enable`. Comment on the benefit of this design choice.

5.9. rising_edge_detector

A1 Report 2

Include a screenshot of the waveforms for this module and briefly explain why they are correct, pointing to specific features in the traces.

5.10. button_hold_detect

A1 Report 2

Include a screenshot of the waveforms for this module and briefly explain why they are correct, pointing to specific features in the traces.

A1 Report 9

Explain why the next-state logic may depend on the output `held` without violating the Moore FSM constraint.

5.11. button_hold_pulse

A1 Report 2

Include a screenshot of the waveforms for this module and briefly explain why they are correct, pointing to specific features in the traces.

A1 Report 10

Explain what this module does. Justify your explanation by referring to the generated waveforms, being precise about timing.

A1 Report 11

Explain how `button_hold_pulse` is a Moore FSM despite `rising_edge_detector` being Mealy.

5.12. button_auto_repeat

A1 Report 2

Include a screenshot of the waveforms for this module and briefly explain why they are correct, pointing to specific features in the traces.

5.13. arming_latch

A1 Report 2

Include a screenshot of the waveforms for this module and briefly explain why they are correct, pointing to specific features in the traces.

5.14. edit_mode_selector

A1 Report 2

Include a screenshot of the waveforms for this module and briefly explain why they are correct, pointing to specific features in the traces.

A1 Report 12

Include your block diagram and explain the combinational logic you used for `enable_counter`, `reset_counter`, and `disarm`.

A1 Additional Oral Preparation 4

Be able to explain the output logic of `edit_mode_selector`.

5.15. editable_counter

A1 Report 2

Include a screenshot of the waveforms for this module and briefly explain why they are correct, pointing to specific features in the traces.

A1 Report 13

Explain why `wire` was used instead of `logic` for the three event signals.

5.16. key_synchroniser

A1 Report 2

Include a screenshot of the waveforms for this module and briefly explain why they are correct, pointing to specific features in the traces.

5.17. decimal_display_driver

A1 Report 2

Include a screenshot of the waveforms for this module and briefly explain why they are correct, pointing to specific features in the traces.

5.18. Board Wrapper (user_top / top_de1_soc)

No waveforms required — verified via the automated test suite only.

A1 Additional Oral Preparation 5

Be able to explain the purpose of a board wrapper and how it is used.

5.19. Settable Watch V1–V4

A1 Report 2

Include a screenshot of the waveforms for this module and briefly explain why they are correct, pointing to specific features in the traces.

(Repeat/adapt for each version, `user_top_watch_v1` through `v4`; for `V4`, select only the traces relevant to the accurate time-setting feature.)

A1 Report 14

Reflect on the role of the test suite: the cost of manual testing on the board, the value of automated regression testing, and what the `CYCLES_PER_SECOND` override implies about designing for testability.

A1 Report 15

(V4) Explain which waveform traces you chose to include and why, given the number of internal signals available.

A1 Report 16

(V4) Does your design cause multiple resets? Why is this not a problem?

6. Programming the Board

A1 Report 17

Reflect on your experience in previous lab classes. Were you typically able to get a design working on the first compile and board test? Which development workflow do you prefer: the VS Code simulation environment, or writing code directly in Quartus?

A1 Report 18

Assume the 50 MHz clock on the DE1-SoC board is accurate to within 50 ppm. How many seconds might your watch drift over a 24-hour period?

Part III.

Assignment 2 — Stopwatch and Timer

7. Formal Verification

7.1. up_down_counter_rst

A2 Report 1

Several approaches exist for extending a module mid-design: modify the original in place; copy it and modify the copy, leaving the original untouched; or copy it, modify the copy, and rewrite the original as a thin wrapper. Can you think of other options? For this specific case, which approach would you take, and how would you name the resulting module(s)? Justify your choice and discuss trade-offs relative to the alternatives.

A2 Report 2

Do the given assertions fully capture the behaviour of an up-down counter with reset? If not, identify any behaviours left unspecified, or any properties a reasonable person might expect that the assertions do not guarantee.

A2 Report 3

The configuration file verifies the counter for three specific values of `MAX`. Since this does not cover all possible values, how would you formally prove correctness when this module is used in a larger design?

A2 Report 4

Include the counterexample waveform from temporarily commenting out `assert (count <= MAX)`. Identify the specific counterexample the checker found and explain why it is one. Then explain why restoring the assertion allows the induction step to succeed.

7.2. cascade_counter

Formal verification only — no additional report items beyond standard conventions.

8. Stopwatch

A2 Report 5

Before reading the suggested design approach, sketch how you would approach the stopwatch design.

A2 Report 6

Using a feature-layering approach, how would you decompose the stopwatch into layers? Describe the responsibility of each layer.

A2 Report 7

Draw a block diagram of the stopwatch design, clearly showing the data path and control signals. Compare it with your initial sketch and comment on any differences.

8.1. stopwatch_counter

A2 Report 8

Explain how your implementation satisfies the first-tick timing constraint: the first centisecond increment must occur one centisecond after `enable` goes high, not sooner.

8.2. snapshot_mux

A2 Report 9

Draw a diagram showing how this is implemented in hardware.

8.3. stopwatch_control

A2 Report 10

Draw the FSM state diagram before implementing it. After completing the implementation, note any differences between your diagram and the final design, and reflect on what caused them.

A2 Additional Oral Preparation 1

The FSM state for `stopwatch_control` can be encoded directly as its output signals rather than using a separate state variable. Be prepared to explain why this works and what advantage it offers.

8.4. user_top_stopwatch_v1

A2 Additional Oral Preparation 2

Be prepared to explain why formal verification is well suited to individual submodules but less suited to integration-level testing, and why simulation is preferred at the integration level.

9. Timer

9.1. `editable_countdown`

A2 Report 11

Find the datasheet for the 74LS193 synchronous 4-bit up/down counter. Using its schematic diagram, explain how the borrow output is generated and why it behaves as it does.

9.2. `user_top_timer_v1`

A2 Report 12

Describe your integration plan: which features did you implement first, and why?

A2 Report 13

The timer requires an FSM to manage its three modes. Explain your FSM design: what are the states, what are the transitions, and how did you encode the states? Justify your choices.

10. Brightness Control

10.1. `user_top_brightness_wrapper`

A2 Additional Oral Preparation 3

Be prepared to explain how PWM is used to control the brightness of a seven-segment display, and why the display appears steady rather than flickering.

A2 Additional Oral Preparation 4

Be prepared to explain the wrapper pattern: what a wrapper module is, how it intercepts signals between two other modules without modifying either, and what advantage this offers when adding new behaviour to an existing design.

11. Integration

11.1. `user_top_timepiece_v1`

A2 Additional Oral Preparation 5

Be sure to understand how packed structs simplify routing large numbers of connections through multiplexers.

11.2. `user_top_brightness_timepiece`

No new report items — verified via `sby` and `pytest`.

12. Synthesis (Yosys Walkthrough)

A2 Report 16

Include `elaborated.png`. Explain what `$lt`, `$add`, and `$mux` compute, and how they work together to implement the counter's next-state logic.

A2 Report 17

Include `proc.png`. What do you think `$sdffe` stands for? Explain its four ports (CLK, D, EN, SRST) in terms of the original `always_ff` block. What does the diagram tell you about what `proc` and `opt` did and did not change?

A2 Report 18

Include `techmap.png`. Why are there now two flip-flop instances rather than one? Why does Yosys slice the wires the way it does? Decode the polarity suffix `PP0P` on `$_SDFFE_PP0P_`.

A2 Report 19

Include `abc.png`. Draw the corresponding circuit from the diagram, then use truth tables to verify it correctly implements the counter's next-state logic for all four states.

13. Enhancements (Optional)

A2 Report 20

If you added extra features, describe what you implemented, how you implemented it, and the reasoning behind any major design decisions.

14. Final Reflection

A2 Report 14

Reflect on your experience designing and implementing a digital watch on an FPGA across both assignments. What went well, what was harder than expected, and what would you do differently if you were starting over?

A2 Report 15

Estimate how much time you saved compared to having used only Quartus. Break down the time savings between better editing features, linting, and automated tests plus formal verification. Without these tools, at what stage would you have discovered a problem, and how much harder would it have been to locate and fix the source of the error?

A2 Report 21

Based on your experience across both assignments, explain which parts of a digital watch are well suited to an FPGA and which would be easier to implement in software. Assume that any component responsible for counting or tracking time requires clock-

A. Cross-Check Index

Use this table to confirm every original report requirement has been covered before submitting.

Original	Assignment	Topic
Report 1	1	Verilator <code>if</code> linting
Report 2	1	Waveform screenshot (every module)
Report 3	1	Backup strategy
Report 4	1	BCD explanation
Report 5	1	<code>\$clog2</code> explanation
Report 6	1	Own test bench, <code>CYCLE_COUNT=1</code>
Report 7	1	Why “PWM generator”
Report 8	1	<code>rst</code> priority benefit
Report 9	1	Moore FSM dependency on output
Report 10	1	<code>button_hold_pulse</code> behaviour
Report 11	1	Moore despite Mealy sub-block
Report 12	1	<code>edit_mode_selector</code> block diagram
Report 13	1	<code>wire</code> vs <code>logic</code> choice
Report 14	1	Test suite reflection
Report 15	1	V4 waveform trace selection
Report 16	1	Multiple resets in V4
Report 17	1	Board vs. simulation workflow
Report 18	1	Clock drift calculation
Report 1	2	Module-extension approach
Report 2	2	Assertion completeness
Report 3	2	Proving correctness for all <code>MAX</code>
Report 4	2	Induction counterexample
Report 5	2	Initial stopwatch sketch
Report 6	2	Feature-layering decomposition
Report 7	2	Stopwatch block diagram
Report 8	2	First-tick timing
Report 9	2	Snapshot mux diagram
Report 10	2	Stopwatch control FSM diagram
Report 11	2	74LS193 borrow output
Report 12	2	Timer integration plan
Report 13	2	Timer FSM design
Report 14	2	Two-assignment reflection
Report 15	2	Time-saved estimate
Report 16	2	<code>elaborated.png</code>
Report 17	2	<code>proc.png</code>
Report 18	2	<code>techmap.png</code>
Report 19	2	<code>abc.png</code>
Report 20	2	Enhancements (optional)
Report 21	2	FPGA vs. software suitability